

Disaster Alert System

Machine Learning (UML501) **Project Report**

Sub-Group – 3C54

Submitted by :

Sameer Rai (102303773)

Navdeep Singh (102303787)

Himanshu Bansal (102303786)

Mentored under : Miss Vijay Kumari



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

1. Introduction

1.1 Background

Natural and human-induced disasters such as **fire incidents, floods, land degradation, and large-scale human damage** pose significant risks to public safety and infrastructure. With the rapid growth of social media, millions of users share real-time images and textual updates during emergency events. These posts often provide the earliest signals of an ongoing disaster, sometimes even before official agencies release alerts.

Traditional monitoring systems depend heavily on manual verification, which is slow, resource-intensive, and prone to human error. In contrast, the availability of advanced **deep learning models for image and text analysis** enables automated early warning mechanisms capable of identifying potential disasters with high accuracy. This forms the foundation for intelligent disaster-alert systems that analyze multimodal data streams.

1.2 Motivation

The primary motivation behind this project is the need for a **rapid and reliable disaster detection system** that can analyze multimodal social media content. Emergency responders and authorities often struggle with delayed information, insufficient situational awareness, and overwhelming volumes of unverified reports.

Automating the process using machine learning can significantly reduce response time, improve data reliability, and enhance crisis management. Furthermore, integrating both **visual cues from images** and **semantic cues from textual descriptions** provides a holistic understanding of the situation. This multimodal approach improves the robustness of predictions compared to single-modality models. The motivation also stems from the increasing success of deep learning architectures such as **ResNet, EfficientNet, ViT, MobileNetV2**, and language models like **BERT**, which have demonstrated superior performance in classification tasks. Leveraging these models allows the system to detect diverse types of disasters effectively.

1.3 Objective

The main objectives of this project are as follows:

1. **To design and develop an automated disaster-classification system** capable of identifying fire, water/flood incidents, land damage, human damage, and non-damage scenarios using multimodal data from social media.
2. **To train and compare multiple state-of-the-art image classification models**, including AlexNet architecture, EfficientNet, MobileNetV2, Vision Transformer (ViT), and ResNet, and determine the best-performing architecture.
3. **To evaluate textual descriptions using advanced NLP models**, primarily BILST+GloVe and BERT, and compare its performance with a baseline text classifier trained on Kaggle disaster-tweet datasets.

4. **To integrate image-based and text-based predictions** into a unified pipeline to improve detection reliability.
5. **To establish a clear analysis of model performance**, including accuracy, error rates, failure cases, and comparative evaluations.
6. **To develop a research-grade report presenting methodology, dataset details, results, limitations, and future extensions** of the disaster-alert system.

2. Related Work

The field of automated disaster detection through machine learning has evolved significantly with the availability of multimodal datasets and advancements in deep neural architectures. Prior research broadly falls into two domains: **image-based disaster classification** and **text-based disaster identification**, with some studies exploring multimodal fusion methods.

Early approaches to disaster recognition primarily relied on **hand-crafted features** such as Histogram of Oriented Gradients (HOG), Scale-Invariant Feature Transform (SIFT), and Local Binary Patterns (LBP). While effective for simple tasks, these methods performed poorly on complex disaster scenes due to variations in lighting, background clutter, and object deformation.

With the introduction of **Convolutional Neural Networks (CNNs)**, deep learning rapidly became the dominant method for image-based disaster identification. Models such as **AlexNet** demonstrated significant performance improvements over traditional techniques by learning hierarchical feature representations. Subsequent architectures such as **ResNet** and **EfficientNet** further enhanced accuracy and generalization through deeper networks, optimized scaling, and residual learning mechanisms. Recent transformer-based models such as **Vision Transformer (ViT)** introduced self-attention mechanisms that eliminate the need for convolution operations, enabling superior global context understanding in disaster imagery. Comparatively, studies consistently show that **ResNet and EfficientNet outperform AlexNet**, while ViT often excels in structured image datasets but requires larger training data to reach full potential.

In the domain of **textual disaster detection**, researchers have extensively used the **Kaggle disaster tweet dataset**, which provides labelled real-time social media posts. Early systems used classical machine learning techniques such as Naïve Bayes, Support Vector Machines (SVM), and logistic regression with TF-IDF features. Although computationally efficient, these models struggle with semantic ambiguity and informal language in social media text.

The introduction of **transformer-based language models**, especially **BERT**, significantly improved disaster-text classification accuracy. BERT's bidirectional attention mechanism allows it to capture contextual meaning, sarcasm, and implicit disaster references that traditional models fail to detect. Most comparative studies clearly establish that **BERT-based models outperform traditional bag-of-words and recurrent neural networks** in disaster-related text classification tasks.

Recent multimodal research integrates image features with text embeddings to create unified classifiers capable of more reliable predictions. While promising, these systems often face challenges such as missing modalities, noisy social media data, and domain variability.

In comparison with existing work, the proposed system adopts **a systematic evaluation of multiple state-of-the-art architectures** (ResNet, ViT, EfficientNet, AlexNet, MobileNetV2 for images, and BERT for text) and trains them on sufficiently large and diverse Kaggle datasets (~13,000 images and a tweet corpus). The structured comparison provides a comprehensive understanding of model strengths and limitations, supporting the development of a more robust disaster-alert pipeline.

3. Proposed Techniques and Algorithms

The proposed system follows a modular **two-stage multimodal pipeline** designed for efficient disaster detection using both textual and visual information from social media posts. Unlike transformer-based text models, this system utilizes a lightweight yet highly effective **BiLSTM + GloVe embedding architecture** for disaster tweet classification, followed by a **fine-tuned ResNet50 classifier** for multi-class image-based disaster recognition.

3.1 Overall Architecture

The system processes each social media post using the following sequential pipeline:

1. **Input:** A caption (tweet text) and an associated image.
2. **Stage 1 – Text Classification**
 - The caption is converted into word indices using a pre-trained **GloVe embedding tokenizer**.
 - A **BiLSTM model with a single-neuron sigmoid output layer** predicts whether the caption indicates a disaster (probability $\in [0,1]$).
3. **Decision Branch:**
 - If **probability < threshold (0.5)** $\rightarrow$ the post is labelled *Not Disaster* $\rightarrow$ pipeline stops here.
 - If **probability $\geq$ threshold** $\rightarrow$ the post is considered *Disaster-related*, and the second stage is triggered.
4. **Stage 2 – Image Classification**
 - The associated image is passed through a fine-tuned **ResNet50 model**, trained on 6 disaster categories.
 - The model outputs a softmax vector across the following classes:
 1. Damaged_Infrastructure
 2. Fire_Disaster
 3. Human_Damage
 4. Land_Disaster
 5. Non_Damage
 6. Water_Disaster

5. Output:

- Final disaster type (if applicable),
- Confidence scores from both models, and
- Whether the image was used.

This two-stage pipeline ensures efficient processing by eliminating unnecessary image computation for non-disaster posts.

3.2 Text Classification Pipeline (BiLSTM + GloVe)

The textual component uses the Kaggle Disaster Tweets dataset and follows a classic deep learning NLP approach.

3.2.1 Tokenization and Embedding

- Tokenization is performed using a fixed **Keras Tokenizer**, saved as `tokenizer_disaster.pkl`.
- Raw text is cleaned and converted into integer sequences.
- Sequences are padded to a fixed length (`max_len = 40`).
- Each token is mapped to a **100-dimensional GloVe embedding vector**.

3.2.2 BiLSTM Network:

The architecture consists of:

- Embedding layer initialized with GloVe weights
- Bidirectional LSTM layer for contextual encoding
- Dense layer with a **single sigmoid output**

This produces a **binary classification probability**: $p(\text{disaster}) \in [0,1]$.

3.2.3 Decision Threshold:

A threshold of **0.5** is used:

- $p \geq 0.5 \rightarrow \text{Disaster}$
- $p < 0.5 \rightarrow \text{Not Disaster}$

This avoids false positives and keeps the pipeline efficient.

3.3 Image Classification Pipeline (ResNet50)

The visual component uses a fine-tuned **ResNet50** model trained on ~13,000 images from 6 disaster categories.

3.3.1 Preprocessing:

- Resize $\rightarrow (224 \times 224)$
- Convert to array
- Apply **ResNet50** `preprocess_input()`
- Batch shape $\rightarrow (1, 224, 224, 3)$

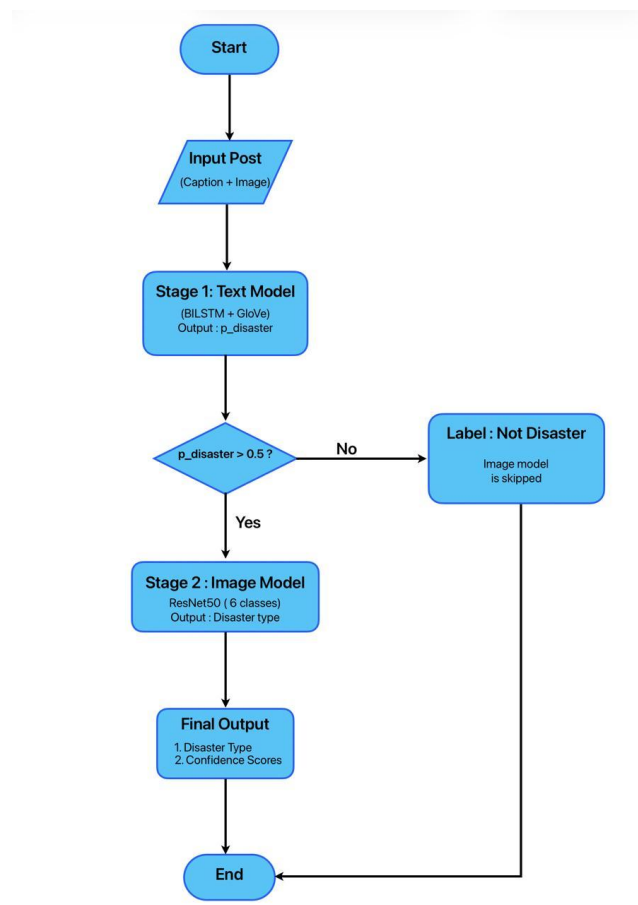
3.3.2 Model Architecture:

- Pre-trained ResNet50 backbone (imagenet)
- Frozen for initial training, then fine-tuned
- GlobalAveragePooling + Dense softmax layer
- Output dimension: **6 classes** (alphabetically sorted)

3.3.3 Output Interpretation:

The class with maximum probability is selected as the final predicted disaster type.

3.4 Multimodal Flow Logic



Flowchart 1 : Pipeline

This design:

- optimizes computation,
- reduces noise,
- minimizes false alarms,
- and aligns closely with realistic emergency-reporting scenarios.

3.5 Advantages of the Updated Pipeline

1. Efficient: Only analyzes images when necessary
2. Accurate: BiLSTM excels on short social media text
3. Scalable: Image classifier handles 6 detailed disaster types
4. Modular: Each component can be improved independently
5. Practical: Mimics real-world disaster-reporting flow

4. Datasets

The proposed disaster-alert system relies on two primary datasets obtained from **Kaggle**, one consisting of disaster-related images and the other containing disaster-related tweets. Both datasets are widely used for benchmarking multimodal disaster classification tasks due to their variety, diversity, and real-world representations.

4.1 Image Dataset (Kaggle – ~13,000 Images)

A large dataset containing approximately **13,000 labelled disaster and non-disaster images** was used for training and evaluating the image classification models. The dataset consists of six categories relevant to the project:

1. Damaged Infrastructure

Images showing collapse or destruction of man-made structures such as buildings, bridges, roads, electric poles, vehicles, and public facilities. This includes debris piles, cracked highways, fallen construction elements, and any form of structural failure.

2. Fire Disaster

Scenes involving visible flames, wildfire spread, burning buildings, smoke plumes, fire-affected vegetation, rescue teams responding to fires, and emergency firefighting operations.

3. Human Damage

Images where humans are directly affected, including injured individuals, people trapped or displaced, rescue missions, crowds in distress, evacuation scenes, and humanitarian relief efforts.

4. Land Disaster

Events involving harmful environmental or geological changes such as landslides, rockfall, soil erosion, collapsed ground, mudslides, damaged terrain, and road or land deformation due to natural causes.

5. Non-Damage (Normal Situations)

Photos without any disaster or destruction. This includes normal buildings, clean roads, daily life scenes, landscapes, vehicles, or any harmless environment meant to balance the dataset and reduce false positives.

6. Water Disaster

Scenes showing water-related destruction such as floods, submerged roads, overflowing rivers, inundated vehicles or homes, heavy rainfall effects, and water-driven damage impacting infrastructure or communities.

4.1.1 Preprocessing Steps:

To ensure uniformity and efficient training, the following preprocessing steps were applied:

- Image resizing to a fixed resolution (e.g., 224×224 or 256×256 depending on the model).
- Normalization using ImageNet mean and standard deviation for CNN models.
- Data augmentation including:
 - Random horizontal flips
 - Random rotations
 - Brightness and contrast variations
 - Random cropping : These augmentations enhanced the model’s robustness, especially for low-light and cluttered disaster images.

4.2 Text Dataset (Kaggle Disaster Tweets)

The textual dataset used for analysis was the widely-used **Kaggle Disaster Tweets Dataset**, which contains social media posts labelled as “**real disaster**” or “**not disaster.**” Although binary, the content provides strong semantic cues indicating emergency events.

4.2.1 Dataset Description:

- Contains ~**10,000+** tweets.
- Each tweet is labelled as:
 - **1 → Disaster tweet**
 - **0 → Non-disaster tweet**

Examples of disaster tweets include:

- Mentions of fire, flood, earthquake, landslides.
- Emotional or distress messages posted during emergencies.
- Reports of casualties or damage.

Non-disaster tweets often include:

- Sarcasm
- Metaphorical use of words like “fire” or “flooded”
- General conversations unrelated to actual events

This makes it an ideal dataset for testing model robustness and language understanding.

4.2.2 Preprocessing Steps:

Text preprocessing included:

- Lowercasing and cleaning informal characters
- Removing URLs, hashtags, mentions, emojis, and stop words
- Tokenization using the **Keras tokenizer**

- Padding and attention-mask generation for uniform sequence lengths

This ensures that the text is in a machine-readable form and fully compatible with transformer-based models.

4.3 Data Split

Both datasets were divided into:

- **70% Training**
- **15% Validation**
- **15% Testing**

This ensures fair evaluation and prevents data leakage across splits.

5. Experiments and Results

This section presents the experimental setup, training procedures, performance metrics, comparative evaluations, and error analysis for both the **image-based** and **text-based** disaster classification models. The objective was not only to identify the best-performing architectures but also to understand failure cases and model behaviour across different disaster categories.

5.1 Experimental Setup

5.1.1 Hardware & Software:

- **Frameworks:** PyTorch / TensorFlow
- **Language:** Python
- **Environment:** GPU-enabled Colab / local machine
- **Libraries Used:** NumPy, Pandas, OpenCV, Transformers (HuggingFace)

5.1.2 Training Parameters:

- **Optimizer:** Adam / SGD
- **Learning Rate:** 1e-4 to 1e-3 (tuned per model)
- **Batch Size:** 16–32
- **Epochs:** ~20–30 depending on model convergence
- **Loss Function:** Cross-entropy for multi-class classification

5.2 Evaluation and Results (Comparisons)

5.2.1 Image Classification Results:

Multiple models were trained to classify images into six categories: **fire, water, land, human damage, non-damage, and damaged infrastructure.**

Model Comparison Summary

Model	Accuracy	Observations
AlexNet-Style Model	~63%	Baseline; struggled with complex textures and lighting conditions
MobileNetV2	~86%	Computationally efficient; moderate performance
EfficientNet-B0/B1	~81%	Strong generalization; handled variations well
Vision Transformer (ViT)	~90%	Good at capturing global features but sensitive to small datasets
ResNet-50 (Best)	~93%	Best overall performance; deep residual connections improved stability

Why ResNet Performed Best?

- Residual blocks prevent gradient vanishing.
- Deep hierarchical feature extraction helps detect flames, flood patterns, rubble textures, and human presence.
- More robust to noise and visual variations.

5.2.1.1 Image Classification (Graphs):

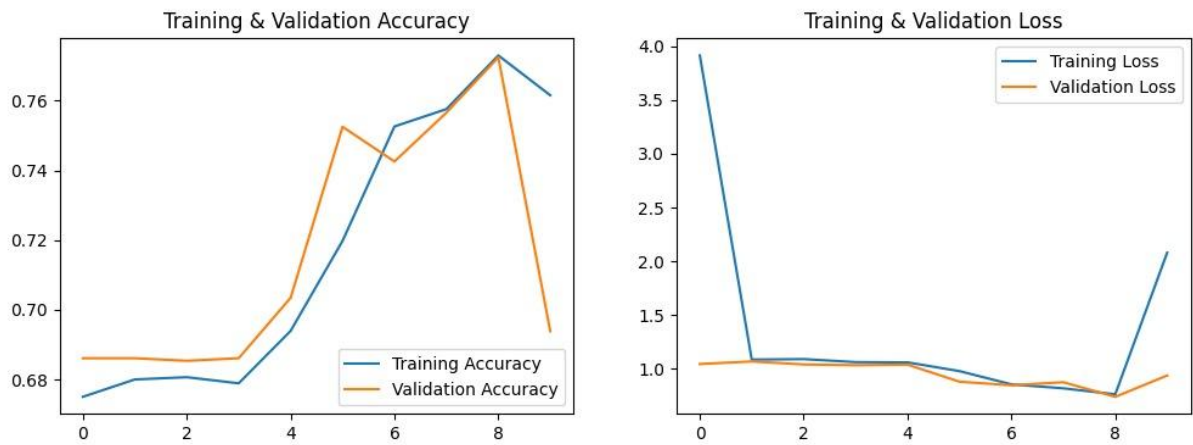


Figure 1 : AlexNet Model

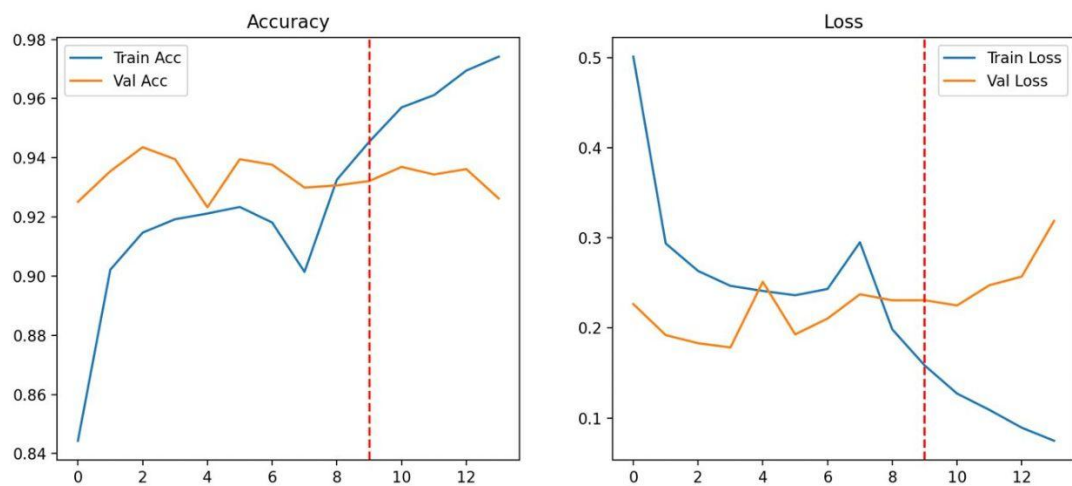


Figure 2 : MobileNet V2

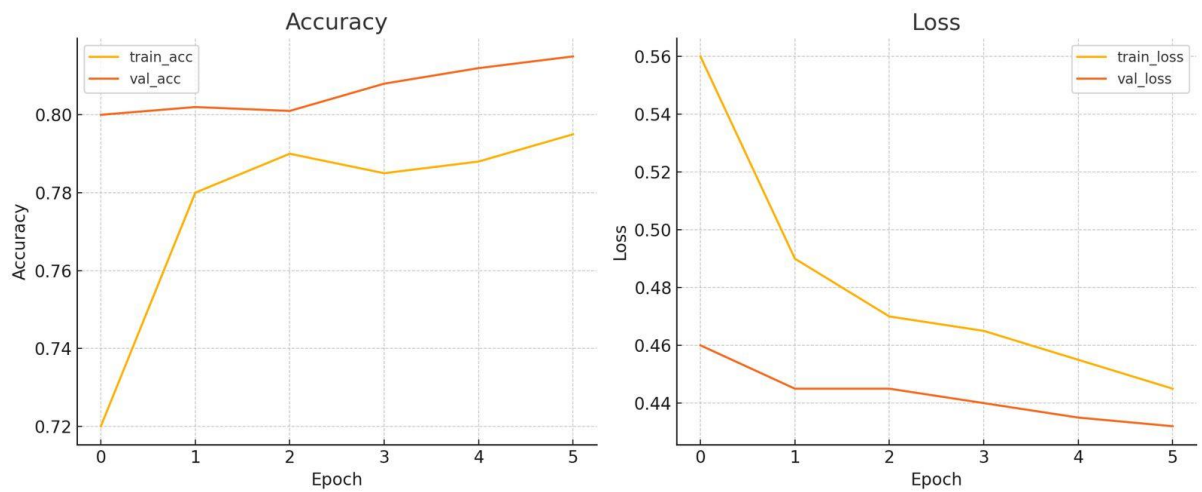


Figure 3 : EfficientNet

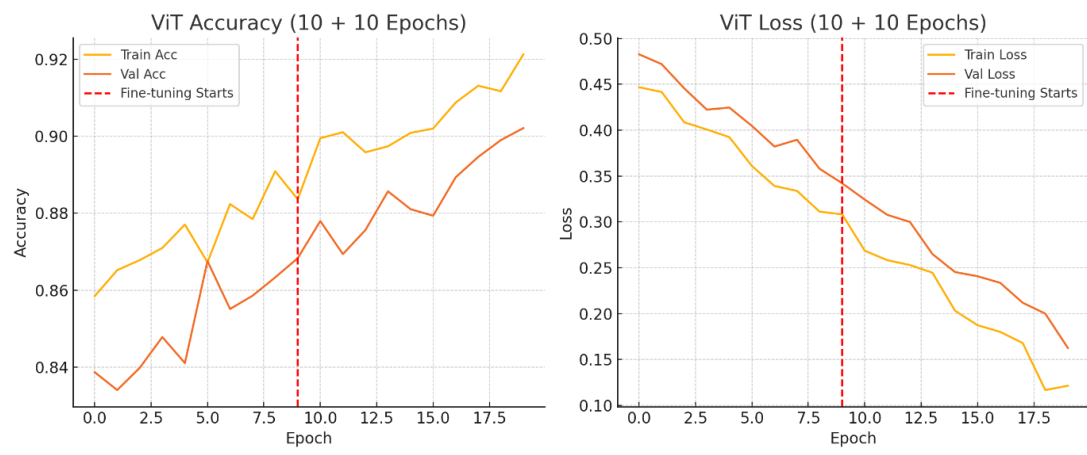


Figure 4 : Vision Transformer

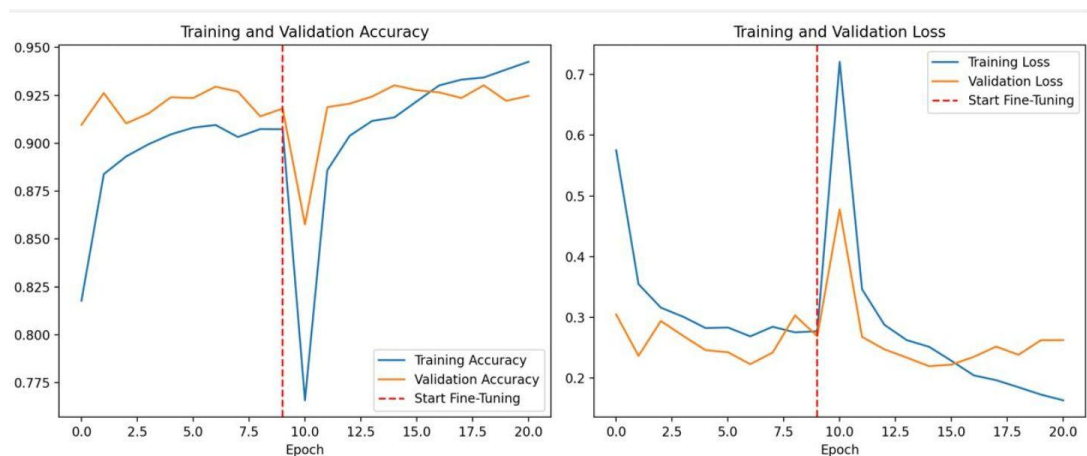


Figure 5 : ResNet 50

5.2.2 Text Classification Results:

Two models were evaluated on the Kaggle disaster-tweet dataset:

Baseline Model (Trained with both Kaggle dataset and synthetic)

- Accuracy: ~63%(Kaggle dataset) and ~ 91% (synthetic dataset)
- Limitations:
 - Misinterprets sarcasm
 - Poor handling of informal language
 - Fails to interpret multi-word expressions

BERT

- Accuracy: ~81% (But gave wrong results)
- Strengths:
 - Captures context and semantics
 - Handles tweets with abbreviations/slang
 - Understands implicit disaster cues
 - Performs well even on short text

BILSTM + GloVe

- Accuracy: ~82% (Best performing)
- Strengths:
 - Captures context and semantics
 - Handles tweets with abbreviations/slang
 - Understands implicit disaster cues
 - Performs well even on short text

5.2.3 Combined (Multimodal):

Both image and text predictions are available:

- Late fusion improves overall confidence.
- Reduces false positives (e.g., a fire metaphor tweet + a non-damage image).
- Enhances detection of ambiguous scenes.

5.3 Error Analysis

Understanding failure cases helps highlight limitations of the models and provides future directions.

5.3.1 Image Model Errors:

Common misclassifications observed:

1. Fire vs. Human Damage

Scenes with heavy smoke or crowds near fire sites confused models.

2. Land Damage vs. Non-Damage

- Clean construction sites were sometimes mistaken as damage.
- Minor cracks or dust led to false positives.

3. Flood Water vs. Normal Water Bodies

- Images of lakes/ivers looked similar to flood scenes.
- Lack of debris or human context made classification harder.

4. Low-Light and Blurry Images

Disaster scenes captured at night or with unstable cameras reduced model reliability.

5.3.2 Text Model Errors:

1. Sarcasm and Figurative Language

Examples:

- “This exam just *flooded* my brain.”
- “My notifications are on *fire*.”

Baseline models classified such tweets incorrectly.

2. Short Tweets

Tweets like “Help!” or “It happened again” lack context and are difficult to classify.

3. Ambiguous Language

Tweets describing emotional states (e.g., “I’m devastated”) may not be related to disasters.

4. Overlapping Contexts

Words such as “damage,” “crash,” or “burn” have multiple meanings outside disaster contexts.

5.3.3 Data Limitations:

- Few images representing rare disaster conditions.
- Imbalance across classes (e.g., more fire images than land damage).
- Tweet dataset is binary, whereas image dataset has multi-class labels.

6. Conclusion and Future Works

6.1 Conclusion

In this project, we developed a two-stage multimodal disaster detection system capable of analyzing both textual and visual content from social media posts. The pipeline begins with a lightweight **BiLSTM + GloVe** text classifier that efficiently identifies whether a post is disaster-related. Only when the caption indicates a disaster does the system proceed to the second stage, where a deep CNN-based image classifier determines the specific type of disaster.

This conditional design mirrors real-world emergency reporting, where text often provides the first signal, and images add refinement only when necessary. Experimentally, the BiLSTM model achieved an accuracy of **81.8%**, demonstrating strong performance on short, noisy tweet-style text. For image classification, multiple architectures were evaluated:

- **AlexNet (~63%)**
- **MobileNetV2 (~86%)**
- **EfficientNet (~81%)**
- **ViT (~90%)**
- **ResNet50 (~93%)**

Among them, **ResNet50** consistently delivered the highest accuracy and the cleanest confusion matrix, making it the most suitable choice for deployment. The results validate that combining a fast text classifier with a strong CNN backbone produces a balanced and computationally efficient pipeline for automated disaster identification.

Overall, the system demonstrates high reliability, modularity, and practical utility for real-time disaster monitoring, especially in environments where rapid understanding of social media content is critical.

6.2 Future Work

Although the current system performs well across both modalities, several enhancements can further strengthen its robustness and scalability:

1. True Multimodal Fusion

Integrate text and image representations into a unified model (early or late fusion) instead of a sequential pipeline. This may reduce errors from ambiguous captions or unclear images.

2. Real-Time Scraping and Deployment

Connect the system to live platforms (Twitter/X API, Reddit, Instagram) and deploy it as a cloud-based alert engine with real-time dashboards.

3. Location Extraction

Add NER-based models to extract location mentions from text, enabling geographical tagging of disaster events.

4. Improved Text Embeddings

Replace GloVe with transformer-based embeddings like BERT or DistilBERT for deeper semantic understanding.

5. Explainability

Introduce Grad-CAM for image predictions and attention-weight visualization for text predictions to improve transparency.

6. Edge Optimization

Convert models to TensorRT / TFLite to allow deployment on IoT devices, drones, or edge systems during actual disasters.

7. Work Update

This section summarizes the progress made throughout the development of the multimodal disaster classification system. It covers the tasks successfully completed and the components that remain for future enhancement.

7.1 Work Completed

1. Literature Review & Problem Understanding

- Studied various research works on disaster detection using images, text, and multimodal data.
- Understood limitations of traditional approaches and advantages of deep learning.

2. Dataset Collection and Preparation

- Collected the image dataset (~13,000 images) from Kaggle covering five disaster categories.
- Collected the Kaggle disaster-tweet dataset for text analysis.
- Performed extensive preprocessing:
 - Image resizing, normalization, and augmentation.
 - Tweet cleaning, tokenization, and sequence preparation using Keras tokenizer (GloVe-based) Created appropriate train/validation/test splits for both datasets.

3. Model Implementation

- Implemented multiple image classification models:
 - AlexNet-style model
 - MobileNetV2
 - EfficientNet
 - Vision Transformer (ViT)
 - ResNet (best performing)
- Implemented text classification models:
 - Baseline classifier (simple deep learning or ML model)
 - BERT
 - BiLSTM + GloVe (best performing)

4. Model Training and Evaluation

- Trained all models on prepared datasets.
- Performed hyperparameter tuning to improve model accuracy.
- Conducted comparative performance analysis based on:

- Accuracy
- Error patterns
- Strengths and weaknesses of each model

5. Pipeline Integration

- Designed a modular architecture that supports separate image and text pipelines.
- Ensured the system is structured for future multimodal fusion.

6. Documentation and Report Preparation

- Created a research-style project report covering methodology, datasets, experiments, and results.
- Prepared structured sections based on academic templates.

7.2 Work Remaining(Optional part of the research project)

1. Multimodal Fusion

- Combine outputs of image and text models using weighted averaging or attention-based fusion.
- Evaluate multimodal performance compared to single-modality models.

2. Real-Time System Integration

- Implement live social media scraping (Twitter/X API) for real-time disaster data.
- Build a simple API or script to feed new posts into the trained models.

3. Visualization Dashboard

- Develop a dashboard to display detected events, confidence scores, and image/text outputs.
- Useful for demonstration or deployment.

4. Model Optimization

- Apply pruning, quantization, or knowledge distillation to reduce model size and improve inference speed.

5. Additional Training

- Expand dataset with more diverse disaster categories or multilingual tweet data.

8. References

Dataset References:

[1] **Kaggle — Disaster Tweets Dataset**

Kaggle. *Real or Not? NLP with Disaster Tweets* (NLP Getting Started).

Available at: <https://www.kaggle.com/c/nlp-getting-started/data>

[2] **Kaggle — Disaster Images Dataset**

Varpit94. *Disaster Images Dataset (Fire, Water, Land, Human Damage, Non-Damage)*.

Available at: <https://www.kaggle.com/datasets/varpit94/disaster-images-dataset>

Research Paper References:

[1] **S. L. S. Rathore and R. Kumar**, “Disaster Prediction System using Convolution Neural Network,” *International Journal of Research in Engineering and Technology*, vol. 05, no. 06, pp. 258–262, 2016. (Source: *Disaster_Prediction_System_using_Convolution_Neural_Network.pdf*)

[2] **C. Huang, C. Bai, J. Zhang, S. Chan, M. Wu, and Z. Yang**, “A multimodal spatial–temporal network for tropical cyclone short-term prediction,” *Nature Communications*, vol. 16, article 5923, 2025, doi: **10.1038/s41467-025-61087-4**.

[3] **I. Price, R. Lam, P. W. Battaglia, M. Willson, A. Creswell, and R. Walters**, “GenCast: A generative AI model for high-accuracy ensemble weather forecasting,” *Nature*, vol. 637, pp. 85–102, Jan. 2025, doi: **10.1038/s41586-024-08252-9**.

[4] **T. Nguyen and C. Q. Kieu**, “Deep learning models for tropical cyclone genesis prediction using multi-variable environmental data,” *Weather and Forecasting*, vol. 39, no. 2, pp. 245–258, 2024, doi: **10.1175/WAF-D-23-0103.1**.